Before WebSocket, web browsers didn’t have any standard way to have data pushed to them. Instead, various tricks were cobbled together to get real-time effects in browsers. Recently, work to standardize the WebSocket protocol (http://tools.ietf.org/html/rfc6455) has made life easier for you, and you can open a connection without much fuss:

```
socket = new WebSocket 'ws://www.agtronsapi.com/server-data-socket'
```

As with most things in a browser, `WebSocket` is event driven, so you must listen for events:

```
socket.onmessage = (message) -> console.log "Received message #
{message}"
```

The server is now responsible for *pushing* the data to the client instead of the client having to ask the server for new data. This is a browser-based program, though, so there must be a catch. Indeed there is.

WebSocket is a new technology, so you can’t be sure that any given browser will support it. Cross-browser compatibility is something you must always be aware of when building browser-based programs. Such programs have to work in multiple different browsers—browsers that may not support the same features.

11.3. Cross-browser compatibility

Imagine now that Scruffy tells you he received an email from his 106-year-old great-grandmother telling you that the dashboard doesn’t work for her. You’re not sure why Scruffy’s 106-year-old great-grandmother needs to use Agtron’s dashboard, but apparently she does. When you ask what she sees when she opens the dashboard, she tells you it’s a blank screen. Why does she see a blank screen?

The problem is that Scruffy’s elderly great-grandmother is using an elderly browser. Suppose this browser is called *Browser X*. When you obtain a copy of Browser X and use it to view the dashboard, you quickly see that there’s an error when it executes `document.querySelector`: